TP5 : Réseau et forges

Introduction

Pour ce dernier TP sur le réseau et les forges sociales, je vous propose de refaire le monde en vous livrant à un petit exercice d'édition collaborative autour de la Déclaration des Droits de l'Homme et du Citoyen de 1789.

Pourquoi ce texte, me direz-vous ? Parce que c'est un texte court, très connu, dont le copyright a expiré depuis longtemps, et dont la nature politique se prête naturellement à des polémiques bon enfant. En un mot, un candidat idéal pour explorer les subtilités de la gestion de version collaborative.

Préparations au travail en réseau

Authentification SSH

Il ne suffit pas de pouvoir vous connecter à l'interface web d'une forge pour pouvoir échanger des *commits* avec. Vous devez également fournir à votre utilitaire git local un moyen de confirmer votre identité auprès du serveur, lors de certaines opérations sensibles comme écrire des *commits* dans un dépôt ou lire les *commits* d'un dépôt privé. On parle de s'authentifier.

Il existe plusieurs moyens de faire cela. Dans ce TP nous utiliserons l'authentification SSH, qui est la plus utilisée. Mais vous pouvez consulter la section « Informations complémentaires » pour plus d'informations sur l'authentification HTTPS, qui est parfois utilisée comme alternative.

Le protocole SSH est basé sur la cryptographie à clé publique, qui utilise une paire de clés cryptographiques :

- La clé privée est une information réputée secrète, qui sert à vous identifier.
- La clé publique doit être transmise aux services auprès desquels vous souhaitez vous identifier au moyen de votre clé privée. Elle leur permet de vérifier que vous possédez la clé privée, mais pas d'usurper votre identité auprès d'autres services ayant connaissance de votre clé publique.

La machine que vous utilisez ne possède pas encore de clés SSH...

```
@ ls ~/.ssh
```

... mais vous pouvez facilement en générer une paire avec la commande suivante :

```
$ ssh-keygen -t ed25519 -C "formation-git"
```

`ssh-keygen` va vous poser une série de questions :

- L'endroit où vous souhaitez stocker vos clés. Je vous recommande d'utiliser l'emplacement par défaut en appuyant simplement sur Entrée.
- La phrase de passe que vous souhaitez utiliser pour protéger votre clé. Pour simplifier la formation, je vous recommande de ne pas utiliser de phrase de passe, en appuyant deux fois de suite sur Entrée.

Ce second point mérite plus ample discussion. Sans phrase de passe, votre clé privée est stockée dans votre dossier personnel sans chiffrement supplémentaire, ce qui est problématique si vous vous retrouvez dans un des cas suivants :

- Votre ordinateur est volé alors que votre disque dur n'était pas chiffré (rare au CNRS, où le chiffrement des postes professionnels est obligatoire).
- Vous partagez votre ordinateur avec d'autres utilisateurs qui ont accès à votre dossier personnel et ne sont pas dignes de confiance.
- Vous exécutez (accidentellement, s'entend) des logiciels malveillants qui s'empressent d'envoyer une copie de votre clé privée à un tiers malveillant.

Toutefois, les méthodes permettant de protéger votre clé SSH de ce type d'attaque sans se retrouver à saisir une phrase de passe à chaque utilisation de git sont un peu complexes et dépendent du système d'exploitation utilisé.

Pour simplifier cette formation, qui est effectuée dans un cadre contraint et pour une durée limitée, à l'issue de laquelle votre machine virtuelle et la clé SSH qu'elle contient seront détruites, nous accepterons donc le risque de travailler sans phrase de passe.

Une fois votre paire de clés SSH générée avec `ssh-keygen`, il ne vous reste plus qu'à afficher votre clé publique, par exemple avec l'éditeur `gedit`...

```
$ gedit ~/.ssh/id_ed25519.pub
```

... puis à la copier-coller dans l'interface web de gestion de clé SSH de GitHub, située sur https://github.com/settings/ssh/new. Je vous recommande de lui donner un nom explicite du style « Formation Git » et de la supprimer après la formation.

Et voilà, vous êtes prêt à échanger des *commits* avec la forge GitHub !

Fork et clonage

Une copie de la Déclaration des Droits de l'Homme et du Citoyen est disponible sur le dépôt git https://github.com/HadrienG2/ddhc.

Pour commencer, créez-en un *fork* sous votre contrôle, en utilisant le bouton « Fork » situé en haut à droite de l'interface de GitHub, entre le bouton « Watch » et le bouton « Star ». Acceptez les paramètres proposés par défaut avec le bouton « Create fork ».

L'opération va prendre un petit moment (typiquement quelques secondes). Lorsqu'elle est terminée, vous devriez être redirigé automatiquement sur votre *fork* nouvellement créé.

Pour travailler dessus localement, vous allez ensuite devoir copier son contenu sur votre machine. Pour cela, vous avez besoin de son URL SSH, que vous pouvez obtenir de l'interface GitHub de la façon suivante :

- Cliquez sur le bouton bleu « Clone ».

- Cliquez sur l'onglet « SSH ».
- Copiez l'URL qui s'affiche alors.

Une fois l'URL obtenue, créez un clone local du *fork* en passant cette URL en paramètre de la commande `git clone` :

```
$ cd ~/Desktop/
? git clone <url de votre fork>
```

Votre client git vous avertira alors qu'il ne connaît pas l'identité du serveur auquel vous vous connectez, ce qui est normal ici puisque c'est la première fois que votre compte utilisateur sur les machines d'enseignement se connecte à GitHub via SSH. Vous pouvez donc autoriser l'opération.

Connexion au dépôt officiel

Comme notre clone local dans le TP précédent, le clone du *fork* possède une *remote* pointant vers le dépôt distant qui permet d'échanger facilement des commits avec ce dernier :

```
$ cd ddhc/
$ git remote -v
```

Mais pour appliquer la méthode de travail basée sur les *forks* et les *pull requests* (appelée *GitHub flow*), il faut aussi être capable d'échanger des commits avec le dépôt officiel, celui à partir duquel vous avez créé votre *fork*. En effet, des nouveautés vont arriver sur ce dernier au fur et à mesure du TP, et vous devrez vous synchroniser avec ces dernières.

Par convention, la *remote* associée au dépôt officiel est souvent appelée *upstream* :

```
$ git remote add upstream https://github.com/HadrienG2/ddhc.git
$ git fetch upstream
```

Notez qu'on peut ici utiliser une URL HTTPS, sans avoir besoin de saisir un mot de passe, puisqu'on ne fait que lire des *commits* depuis un dépôt public, ce qui ne nécessite pas de s'identifier.

Si vous ouvrez maintenant `gitg`, vous pourrez constater que vous pouvez maintenant visualiser vos commits et branches locales, celles de votre *fork*, et celles du dépôt officiel. Tout est prêt !

Travail distribué avec le *GitHub flow*

Introduction

Le *GitHub flow* est une méthode de travail qui, malgré son nom, n'a rien de spécifique à la forge GitHub. Toutes les forges sociales modernes peuvent être utilisées de cette manière. Dans cette méthode, le travail s'organise autour de quatre grandes étapes :

1. Rester à jour par rapport au dépôt officiel
2. Préparer des changements dans une branche dédiée de son dépôt local
3. Publier ses changements sur son *fork* lorsqu'ils sont prêts
4. Les soumettre au dépôt officiel pour qu'ils y soient intégrés

Examinons ces quatre étapes en détail

Se synchroniser avec le dépôt officiel

Comme nous l'avons vu dans le TP précédent, il est possible de récupérer les changements du dépôt officiel avec `git fetch` :

```
$ git fetch upstream
```

Cependant, dans le cadre de cette méthode de travail, nous n'avons généralement pas besoin de nous intéresser à l'ensemble des branches de ce dépôt. Il suffit de tenir notre branche `master` à jour, ce que l'on peut faire avec `git pull` après avoir configuré `upstream/master` comme branche amont :

```
$ git checkout master
$ git branch --set-upstream-to=upstream/master
```

Par la suite, vous pourrez mettre à jour votre master via :

```
$ git checkout master
$ git pull --ff-only
```

Je conseille ici l'utilisation de `--ff-only` pour vous aider à repérer une erreur classique. Si l'on applique correctement le *GitHub flow*, la branche `master` du dépôt officiel devrait avancer de façon monotone, et votre branche `master` locale devrait suivre son avancée. Par conséquent, si vous ne pouvez pas mettre en avance rapide, cela relève d'un comportement pathologique :

- Soit les mainteneurs du dépôt officiel ont fait diverger `master`. C'est parfois nécessaire, mais très rare, et un mainteneur responsable vous informera lorsqu'il doit en venir à de tels extrêmes.
- Soit, cas de loin le plus courant, vous avez vous-même créé une divergence, typiquement en ajoutant des commits sur votre branche `master` locale.

Dans ce dernier cas, pour vous tirer d'affaire, vous pouvez...

- Si vous le souhaitez, sauvegarder vos changements dans une nouvelle branche avec `git branch <nom>`
- Puis forcer votre branche `master` à revenir au même point que le `master` officiel avec la commande `git reset --hard upstream/master`

Vous pouvez configurer l'utilisation implicite de `--ff-only` pour tous les `git pull` futurs au sein de ce dépôt avec la commande de configuration...

```
$ git config pull.ff only
```

Construire des changements dans une branche dédiée

Une fois que l'on a un `master` à jour, il est possible de construire des changements par-dessus ce dernier. Il suffit pour cela de créer une branche qui part de master et de commencer à créer des *commits* dessus. En voici **un exemple**, à adapter selon le type d'édition que vous voulez effectuer :

```
$ git checkout master
? git checkout -b changements
? echo -e "\nCeci n'est pas un pied de page" >> README.md
$ git add README.md
? git commit -m "Ajout d'un pied de page rebelle"
```

Mais pendant que vous effectuez vos modifications, le dépôt officiel continue d'évoluer, et il est possible que des nouveautés arrivent sur la branche `master`. Les sorties des commandes `git fetch` et `git pull` vous en informeront.

Si c'est le cas, vous pourrez intégrer ces nouveautés à votre branche de développement avec les outils que nous avons vus précédemment. Un cycle complet de mise à jour de `master` et d'intégrations de ses nouveautés à votre branche pourrait ainsi ressembler à ceci :

```
$ git checkout master
$ git pull --ff-only
? git checkout changements
$ git merge master
```

En cas de conflit de fusion, il vous appartiendra de vous adapter aux nouveautés de la branche `master` officielle qui sont incompatibles avec vos changements.

En règle général, plus une de vos branches a une durée de vie longue, plus il y a de chances que cela se produise. De plus, si vous utilisez `git rebase` plutôt que `git merge`, comme la politique de certains projets l'exige, il peut aussi arriver que vous rencontriez de conflits de fusion en cascade, où vous devrez faire face à des conflits non pas une seule fois, mais une fois par commit de votre branche !

Pour éviter cela et simplifier le passage en revue de vos changements par les mainteneurs, il est recommandé de privilégier des branches de durée de vie courte, aux objectifs précis et modestes, que l'on soumettra au dépôt officiel dès que possible. Nous allons maintenant étudier ce processus de soumission.

Mettre en ligne des changements sur son *fork*

Lorsque vous êtes satisfait de votre branche locale, vous pouvez la mettre en ligne sur votre *fork* avec la commande `git push`.

L'option `--set-upstream`, que nous avons évoquée précédemment, vous permettra de mémoriser l'association entre votre branche locale et la branche présente sur votre *fork*.

```
? git push --set-upstream origin changements
```

Ainsi, vous pourrez par la suite vous contenter d'un simple `git push`.

Nous l'avons évoqué précédemment, dès lors que vous avez mis des changements en ligne sur un serveur, il faut être prudent avec les modifications d'historique et les `git push -f` associés pour ne pas « couper l'herbe sous le pied » aux gens qui travaillent par-dessus vos branches.

Mais l'un des avantages qu'il y a à utiliser un *fork*, c'est que ce dépôt n'appartient qu'à vous. Personne d'autre n'est donc censé baser son travail dessus, et c'est pourquoi la plupart des projets tolèrent l'utilisation libérale du *force-push* sur les *forks*, un confort très appréciable pour les amateurs d'édition d'historique.

Gardez cependant à l'esprit que cette règle de vie est à double tranchant. Si vous devez un jour baser votre travail sur le *fork* de quelqu'un d'autre, il vous appartiendra de vivre avec les suppressions de branches et autres modifications d'historique effectuées par cette personne sur son *fork*. Évitez donc si possible de baser du travail sur le *fork* d'autrui, en privilégiant plutôt le travail à partir du dépôt officiel du projet.

Soumission au dépôt officiel

Une fois que vous avez mis des changements en ligne sous la forme d'une branche sur votre *fork*, vous pouvez demander l'intégration de ces changements au dépôt officiel avec une fonctionnalité des forges git modernes, la *pull request*. Cette fonctionnalité est également appelée *merge request* par GitLab, mais comme nous utilisons GitHub dans ce TP, nous nous en tiendrons à sa terminologie afin d'éviter toute confusion.

Cette possibilité vous sera généralement directement offerte par l'affichage d'une URL dans la console lorsque vous poussez la branche sur votre *fork*, ainsi que par un bandeau dans l'interface web de votre *fork* sur GitHub pendant quelque temps après le *push*. Mais si vous avez manqué ces raccourcis, vous pouvez créer « manuellement » une *pull request* en allant dans l'interface web de votre *fork*, affichant la liste des branches, et cliquant sur le bouton « Pull Request ».

Comme son nom l'indique, une *pull request* invite les mainteneurs du dépôt officiel à intégrer des changements au projet en fusionnant les nouveautés de votre branche dans un historique officiel (généralement la branche `master`). Vous êtes invité à donner un titre court à votre soumission, ainsi qu'une description qui peut être plus étoffée et agrémentée de mise en page Markdown. Selon la forge que vous utilisez, d'autres fonctionnalités de gestion de projet peuvent vous être proposées lors de la création d'une *pull request*, nous vous invitons à consulter la documentation de votre forge pour en savoir plus.

Pour les mainteneurs, une *pull request* se présente sous la forme du titre et de la description que vous avez donnée, d'une visualisation des changements proposés, et d'une interface permettant de discuter desdits changements (pour proposer des modifications avant intégration par exemple).

Il est aussi possible pour le mainteneur de définir une batterie de tests automatisés que toute *pull request* doit passer avant d'être acceptée par le projet. On parle alors d'intégration continue.

Si des changements sont demandés par le mainteneur, vous pouvez ajouter ceux-ci à votre branche locale via de nouveaux commits et de les pousser sur votre *fork* avec un simple `git push`. La *pull request* associée sera alors mise à jour automatiquement.

À vous de jouer !

À ce stade, vous en savez assez pour commencer à vous amuser avec cette pauvre Déclaration des Droits de l'Homme.

Peut-être avez-vous toujours rêvé de paraphraser Olympe de Gouges en réécrivant entièrement le texte au féminin (« Déclaration des Droits de la Femme et de la Citoyenne ») ? Peut-être souhaitez-vous y ajouter l'article manquant qui résoudra enfin le débat séculaire entre pain au chocolat et chocolatine ? Ou bien peut-être est-il temps d'amender l'article 15 qui autorise chaque citoyen à demander des comptes aux agents publics ?

Modifiez le texte comme vous le souhaitez, en n'hésitant pas à créer des commits en cours de route pour vous permettre de revenir facilement en arrière. Puis, quand vous aurez fini, poussez votre branche finale sur votre *fork*, et soumettez une *pull request* au dépôt officiel pour terminer ce TP.

Conclusion

Dans ce dernier TP, nous avons vu comment utiliser git en réseau, avec une méthode de travail basée sur les *forks* et les *pull requests*.

Si ce n'est pas la seule manière d'utiliser git, cette méthode est une des plus populaires aujourd'hui, car elle sécurise le dépôt officiel, facilite les contributions extérieures, et évite certains problèmes liés à l'utilisation naïve d'un dépôt git partagé (par exemple un *force-push* malencontreux sur une branche du dépôt principal qui sème le chaos dans le projet).

Nous terminerons cette formation par une exploration « à la carte » de sujets plus avancés, ou de tout point déjà abordé que vous souhaiteriez approfondir.

Exercices

1. Partez de votre clone du dépôt `ddhc`.
2. Vérifiez que les URLs de vos deux *remotes* sont correctes : `origin` doit pointer sur votre *fork* et utiliser une connexion SSH, `upstream` doit pointer sur le dépôt officiel, et utiliser une connexion HTTPS.
3. Créez une nouvelle branche partant du `master`, et ajoutez-y un commit.

4. Poussez cette branche sur votre *fork*.
5. Ajoutez deux nouveaux commits à la branche, puis poussez-les sur votre *fork* aussi.
6. Mettez à jour votre dépôt vis-à-vis des éventuelles nouveautés du dépôt officiel du projet.
7. Détruisez la version locale de votre nouvelle branche avec `git branch -D`, puis régénérez-la à partir de la version sauvegardée sur votre dépôt.

Antisèche

Ce TP n'est qu'une mise en pratique des enseignements du TP4, et n'introduit aucune nouvelle commande git. Par conséquent, nous vous invitons à vous référer aux TPs précédents pour retrouver les commandes utilisées.

Informations complémentaires

Authentification HTTPS

Vous pourriez être amené au cours de votre carrière à travailler dans un réseau informatique bloquant le trafic SSH sortant. Si cette politique réseau n'est pas négociable, cela empêche l'utilisation du protocole SSH auquel nous avons fait appel pendant ce TP.

À la place, vous devrez utiliser le protocole HTTPS, basé sur un couple nom d'utilisateur + mot de passe. Ceci nécessite quelques étapes préparatoires :

- Cloner votre dépôt git depuis l'URL HTTPS fournie par la forge, ou dans le cas d'un dépôt déjà cloné changer l'URL des *remotes* associées à la forge avec la commande `git remote set-url <remote> <URL>`.
- Pour les forges utilisant un mécanisme d'authentification « SSO » comme gitlab.in2p3.fr...
  - Trouver votre nom d'utilisateur sur la forge (dans le cas de GitLab, il apparaît comme `@<nom>` quand on clique sur le menu en haut à droite).
  - Configurer un mot de passe dédié aux échanges git. Sur l'instance GitLab de l'IN2P3, cela peut être fait à l'adresse suivante : https://gitlab.in2p3.fr/-/profile/password/edit. - Essayez de laisser le champ « Current password » vide, et si ça ne fonctionne pas utilisez le lien « I forgot my password » pour obtenir par mail un mot de passe utilisable.

Une fois ceci fait, vous devrez saisir votre nom d'utilisateur et votre mot de passe à chaque fois que vous écrivez des données sur la forge (via `git push`), et également à chaque fois que vous lisez des données (via `git clone`, `git fetch` ou `git pull`) dans le cas d'un dépôt privé. Ce qui devient rapidement rébarbatif.

Pour une meilleure ergonomie, il est possible de conserver une copie du mot de passe en RAM pour la durée de votre session (sûr) ou de stocker votre mot de passe sur le disque dur (dangereux). Pour savoir comment, référez-vous à la page « gitcredentials » de l'aide de git via `git help gitcredentials` ou https://git-scm.com/docs/gitcredentials.

Petit tour d'horizon des forges logicielles

Dans ce TP, nous avons utilisé GitHub, mais il existe de nombreuses autres forges logicielles basées sur git.

Les trois moteurs de forge les plus utilisés à l'heure actuelle sont GitHub, GitLab et BitBucket. Les éditeurs de ces applications web proposent des instances hébergées sur leurs propres serveurs, selon un modèle commercial « freemium » (niveau de service gratuit limité, pour avoir plus il faut souscrire à un abonnement). Il est également possible d'en héberger une instance sur un serveur que l'on contrôle, par exemple lorsque le secret professionnel l'exige.

Dans ce cadre d'auto-hébergement, GitLab est le moteur le plus populaire dans le secteur académique. On peut mentionner, par exemple, des instances ayant des milliers d'utilisateurs au CERN, au centre de calcul de l'IN2P3 (CCIN2P3), ou au sein du réseau Mathrice (plate-forme PLM), ainsi que de nombreux déploiements plus modestes au sein de laboratoires individuels. La popularité de GitLab tient sans doute au fait que c'est l'éditeur de forge le plus ouvert à l'auto-hébergement.

Parmi les hébergeurs publics, GitHub domine. C'est donc souvent la meilleure solution lorsque l'on souhaite collaborer avec des développeurs hors de son cercle de sociabilité académique, ou lorsque l'on n'a pas accès à un hébergeur « en propre ». Notez toutefois que la légalité de l'utilisation de GitHub, hébergeur américain, pour stocker des données de recherche européennes, est un sujet de polémique récurrent vis-à-vis duquel un consensus clair se fait toujours attendre.

Les moteurs de forge ci-dessus ont de nombreuses fonctionnalités en commun :

- Visualisation de l'historique et des différentes branches
- Parcours de l'arborescence des fichiers à la tête d'une branche donnée
- Visualisation de certains formats de fichier (ex : Markdown, code source)
- Possibilités limitées d'édition de fichiers en ligne
- Gestion de tickets (pour signaler des problèmes, recevoir des demandes…)
- Soumission et revue de changements (*pull requests*/*merge requests*)
- Documentation collaborative au format wiki
- Statistiques diverses et variées sur les contributions
- Outils de gestion de projet agile
- Systèmes de notification élaborées pour « suivre » l'évolution d'un projet

Cependant, chaque moteur a aussi ses spécificités :

- GitLab a toujours fourni en standard des fonctionnalités comme l'intégration continue ou l'hébergement d'images Docker, là où chez

GitHub ce type de service a longtemps été délégué à des tierces parties comme Travis CI. Depuis fin 2019, il existe une solution GitHub « en propre », mais comme on pourrait s'y attendre elle est un peu moins mature que la concurrence au moment où sont écrites ces lignes (mi-2020).
- BitBucket propose une intégration poussée avec d'autres applications web du même éditeur (JIRA, Bamboo, Confluence, HipChat…)
- Le vocabulaire utilisé et le détail des statistiques proposées varie d'une forge à l'autre, et il n'est pas trivial de migrer l'ensemble d'un projet (y compris les tickets, *merge requests*…) d'une forge à une autre.

Autres méthodes de travail

Dans ce TP, nous nous sommes intéressés au *GitHub Flow*, une des méthodes de travail les plus utilisées avec git. Mais c'est loin d'être la seule. On peut mentionner comme autres méthodes classiques…

- Le *GitLab flow*, une variante simplifiée où au lieu d'utiliser des *forks*, on utilise des branches de développement au sein d'un dépôt commun.
- Le *Git flow*, une variante plus sophistiquée où l'on maintient plusieurs branches spéciales au sein du dépôt commun :
  - On distingue `master`/`main`, qui représente les développements publiés, de `develop`, qui contient des développements expérimentaux. Les nouveaux développements sont fusionnés dans `develop`.
  - Avant de sortir une nouvelle version, on crée une branche dédiée (*release branch*) partant de `develop`, qui ne reçoit plus de nouvelles fonctionnalités mais peut encore recevoir des correctifs. Lorsqu'on est satisfait de l'état de cette branche, on la fusionne dans `master` et crée une étiquette pour la nouvelle version officielle.
  - En cas de problème urgent dans `master`, on peut créer une branche `hotfix` issue de `master`, dédiée à la résolution de ce problème et vouée à être immédiatement fusionnée dans `master` à la fin.

Voici un article très complet sur ce sujet qui part des principes fondamentaux pour expliquer comment différentes méthodes de travail en découlent : https://martinfowler.com/articles/branching-patterns.html.